

● PART 2

scikit-learn

Modeling, Evaluation, and Selection

Fit · Predict · Evaluate · Tune · Save

What scikit-learn gives you

1

Preprocessing

split, impute, scale, encode — done in Part 1

2

Supervised learning

regression, classification, ...

3

Unsupervised learning

clustering, dimension reduction, ...

4

Model evaluation

metrics on held-out data

5

Model selection

tune hyperparameters on validation

Model Fitting — supervised

In supervised learning, fitting a model means finding the **best parameters** that **minimize the error** between the model's predictions and the true values on the **training data**.

Once the model is fit on the training data, it can be used to make **predictions on test data**.

Code template — supervised

Assume the data are already processed (Part 1).

```
# Instantiation — import ModelName first  
model = ModelName(...)  
  
# Fitting  
model.fit(X_train, y_train)  
  
# Prediction  
y_pred = model.predict(X_test)
```

Model Fitting — unsupervised

In unsupervised learning, fitting a model means finding the **best parameters** that **best explain the training data itself**.

Once the model is fit on the training data, it can be also used to **test data**.

Code template — unsupervised

Dimension reduction

```
model = ModelName(...)
model.fit(X_train)

X_train_reduced = model.transform(X_train)
X_test_reduced = model.transform(X_test)
```

Clustering

```
model = ModelName(...)
model.fit(X_train)

train_labels = model.labels_
test_labels = model.predict(X_test)
```

*Some dimension reduction methods don't support `.transform()`.
These methods are purely for visualization (EDA), and it can be used as a guideline for your next steps.*

Next

Supervised learning

- Evaluate model
- Select model (tune hyperparameter)
- Save the best model
- Final test: report the results on test set
- Interpret

Unsupervised learning

- Evaluate model (model specific)
- Select model (model specific)
- Save the best model
- Dimension reduction
Use reduced dataset for supervised learning
- Clustering
Use the labels as target variable
Can be used as a guideline

From now on, we will focus on supervised learning only.

Evaluation

Model evaluation is the process of measuring how well a trained model performs on **both the training data and test data**.

```
train_result = EVAL_METRIC(y_train, y_train_pred)
test_result  = EVAL_METRIC(y_test,  y_test_pred)
```


Metrics for regression

Metric	Description	scikit-learn function
MAE	Average of absolute errors	<code>mean_absolute_error</code>
MSE	Average of squared errors	<code>mean_squared_error</code>
RMSE	Square root of MSE (same unit as target)	<code>root_mean_squared_error</code>
R ² score	Proportion of variance explained	<code>r2_score</code>

Metrics for classification

Metric	Description	scikit-learn function
Accuracy	Proportion of correct predictions	<code>accuracy_score</code>
Precision	$TP / (TP + FP)$ — are positive calls right?	<code>precision_score</code>
Recall	$TP / (TP + FN)$ — did we catch the positives?	<code>recall_score</code>
F1 score	Harmonic mean of precision and recall	<code>f1_score</code>
Confusion matrix	Table of TP, FP, FN, TN counts	<code>confusion_matrix</code>
ROC AUC	Area under the ROC curve	<code>roc_auc_score</code>

Accuracy alone is misleading on imbalanced data — pick the metric that matches the cost of each error.

Train vs. test performance

The gap between the evaluation metric on the training/test set tells you what is wrong.

Underfitting

poor on train, poor on test

model too simple

Good fit

good on train, good on test

what we want

Overfitting

great on train, poor on test

model memorized the noise

However, we cannot use the test set for comparison since it will be used only once at the final step, to report the final score!

Overfitting will be detected and controlled on the **validation set**.

When training performance keeps improving while validation performance starts to degrade, that's the **overfitting signal**.

Hyperparameters and validation

- **Hyperparameters are not learned**

they are set by you before training — k in KNN.

- **Goal**

find the values that perform best on unseen data.

- **Grid search**

Provide a list (grid) of hyperparameters and choose (search) one that works best on validation set.

- **Cross-validation**

Instead of using a single validation split (training-validation), we use multiple splits and use the averaged result to find the best hyperparameters.

One advantage of cv is that all the training set can be used as validation set once.

Code template — GridSearchCV

```
from sklearn.model_selection import GridSearchCV

# do NOT set the tuned params here
model = ModelName(...)

param_grid = {
    PARAMETER_1: CANDIDATE_1,
    PARAMETER_2: CANDIDATE_2,
}

grid_search = GridSearchCV(
    estimator=model,
    param_grid=param_grid,
    cv=5,
    scoring='scoring_method',
)
grid_search.fit(X_train, y_train)

best = grid_search.best_estimator_

y_pred = best.predict(X_test)
```

5-fold cross-validation



The scoring strings

The metric you optimize in cross-validation is passed as a string.

Regression

MAE	<code>'neg_mean_absolute_error'</code>
MSE	<code>'neg_mean_squared_error'</code>
RMSE	<code>'neg_root_mean_squared_error'</code>
R ²	<code>'r2'</code>

Classification

Accuracy	<code>'accuracy'</code>
Precision	<code>'precision'</code>
Recall	<code>'recall'</code>
F1	<code>'f1'</code>
ROC AUC	<code>'roc_auc'</code>

“neg_” prefixes exist because GridSearchCV always maximizes the score — larger is better.

Save and load the model

1

Save

after the search, keep the best estimator

```
import pickle

with open('models/ModelName/model.pkl', 'wb') as f:
    pickle.dump(grid_search.best_estimator_, f)
```

2

Load

reuse it anytime, no refitting

```
import pickle

with open('models/ModelName/model.pkl', 'rb') as f:
    model = pickle.load(f)
```

Building several models? Save each best model under its own `models/ModelName/` directory.

● NEXT?

You have a model. Now what?

Interpret

What do the fitted parameters actually say?

Visualize

Residuals, confusion matrix, learning curves.

Communicate

Findings, not code. Who is the audience?

Try other models

Same interface — swap `ModelName` and rerun.

Golden rule, again: fit on train · apply to both · never leak the test set.